



SEPAR: A New Lightweight Hybrid Encryption Algorithm with a Novel Design Approach for IoT

Arsalan Vahi¹ · Somaye Jafarali Jassbi²

Published online: 27 May 2020
© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract

This paper presents a new hybrid encryption algorithm with 16-bit block size and a 128-bit initialization vector, referred to as SEPAR, and it is suitable for IoT devices. The design idea of this algorithm combines pseudorandom permutation and pseudorandom generator functions. This smart integration causes resistance improvement against common cryptographic attacks meanwhile leads to cipher speed increment. Investigation of security analysis on the algorithm and results of the NIST statistical test suit proves its resistance against common cryptographic attacks as linear and differential cryptanalysis. Furthermore, efficient software implementation of SEPAR is presented on 8, 16 and 32-bit platforms. Compared to BORON cipher, SEPAR provides 42.22% throughput improvement on 32-bit ARM CPU. Also, for 8-bit and 16-bit microcontroller, SEPAR provides 87.91% and 98.01% performance improvements compared to present, respectively.

Keywords Lightweight cryptography · Pseudorandom permutation · Pseudorandom generator · Cryptographic attacks · Internet of things

1 Introduction

Technological advancements pave the way for combining environmental objects with the idea of creating a prosperous life. Development of sensor network technologies, coupled with the ever-expanding deployment of embedded systems, which have been symmetrical with the growth of pervasive computing, have enabled the integration of more advanced objects with matured radio frequency identification technology, have made the evolutionary path of IoT. Challenging keys in IoT applications are security reinforcements in confidentiality, privacy, and authentication areas, mainly due to the existing close relationships

✉ Arsalan Vahi
arsalan.vahi2009@gmail.com
Somaye Jafarali Jassbi
s.jassbi@srbiau.ac.ir

¹ Department of Computer Engineering, Science and Research Branch, Islamic Azad University, Tabriz, Iran

² Department of Computer Engineering, Science and Research Branch, Islamic Azad University, Tehran, Iran

between objects and people [1, 2]. Employing the typical security mechanisms does not operate with these objects due to limitations on computing power and energy consumption. By considering these features and limitations, Lightweight cryptography offers as an appropriate solution in this security field. Providing newly-designed Lightweight ciphers for lightweight applications are mainly has two approaches: In the first approach, minor modifications to common ciphers such as DES are applied to provide efficient implementations. A well-known designed cipher of this category is DESL. The main drawback of designed ciphers with this approach is in preserving the cipher's vulnerabilities. As an example, the key length of DESL, which is designed by a slight change to DES, is not adequate for many of today's applications. The second approach is designing a new algorithm based on cope with a trade-off between cost, security, and performance aspects. Well-designed examples of these algorithms are Present, HEIGHT and Boron ciphers [3]. In Present-128, which is a block cipher with a length of 64-bit and a key length of 128-bit, 32 consecutive permutation-substitution operations are performed on the 64-bit input block and the result of 64-bit output is produced [4]. The Present cipher is notable for its compact size and became a standard for lightweight cryptographic methods. It is worth remarking that this cipher has good performance on both hardware and software platforms among other lightweight ciphers. The design idea of the Hummingbird cipher is motivated by the well-known Enigma machine wherein 16-bit input data is merged with four 16-bit values and generates a 16-bit output ciphertext [5]. Cryptanalysis investigations revealed some major weaknesses in this cipher structure as birthday attack. The main reason for susceptibility to this attack lies in the improper trade-off between memory consumption and security, in which security sacrificed to low memory consumption by selecting 64-bit internal states [6].

Symmetric cryptography divides into two categories namely block and stream ciphers. The main idea of block cipher structure is based on pseudo-random permutation. Block cipher design emphasizes on performing successive iterations of permutation- substitution operations on input block. This process provides remarkable security despite its low speed. In contrast to Block ciphers, pseudo-random functions are the fundamental design idea of stream ciphers [7]. Despite the complexity in analyzing and designing process of pseudo-random functions, they can simply be modeled by finite-state machines. These finite-state machines can be utilized as pseudo-random bit generators. Block ciphers have low speed and rather high hardware costs, hence they are inefficient in the environments with interruption, noise, and uncertain data amount properties. The advantages and disadvantages of block and stream ciphers are in contrast to each other in the majority of cases. Data integrity, which is an important security service, is not supported by Stream cipher, however, Block ciphers provide data integrity in addition to confidentiality [7, 8]. Stream ciphers are vulnerable to improper implementations which is the major drawback of their versatile applications. Overall, the performance of Block ciphers and Stream ciphers is efficient on either of the hardware or software platforms, and neither of them has the efficiency in both platforms simultaneously [9, 10]. By considering the diversity of objects in IoT, the requirement for lightweight cryptographic algorithms with high efficiency in both Software and Hardware platforms is essential. In this paper, a new hybrid symmetric cipher, known as SEPAR, is proposed. The fundamental design idea is a smart combination of block and stream ciphers. The design strategy is to cope with the trade-off between security, cost, and performance which leads to a cipher with less power consumption and good cryptographic properties. Concerning the security aspect, choosing the encryption key with 256-bit length ensures the algorithm's security for the majority of usages. From the performance aspect, set the input size and internal operations to 16-bit, applying lightweight math operators

such as XOR and shift, choosing 144-bit internal memory, which is compatible with the majority of limited resource devices, 4×4 S-boxes instead of 8×8 S-boxes, meanwhile leads to cipher efficiency improvement and memory consumption decrement, guarantees the robustness of the proposed scheme to cryptographic attacks such as birthday attack. The proposed cipher performs at lower memory cost and higher performance as compared to other ciphers on 8, 16 and 32-bit software platforms. Also, complete security investigations of cipher prove its resistance against common and important attacks.

The structure of this article is as follows. The design strategy and cipher details are presented in sections two and three. The security analysis of the SEPAR is discussed in Section four. Implementation of the algorithm on software platforms of 8, 16, and 32-bit microcontrollers are provided in section five, and section six, results and concluding remarks are offered.

It is worth noting that the implementation of the algorithm in the hardware platform is discussed in a separate paper.

2 The Design Strategy

The design strategy of SEPAR is to cope with the trade-off between security, cost, and performance [3] (Fig. 1).

In this design strategy, the key length, which varies between 56 and 256, provides a trade-off between cost and security. The hardware architecture, which can be either parallel or serial, provides a trade-off between cost and performance. The number of rounds provides a trade-off between security and performance. These design goals can be assessed by measuring the key length, throughput, and memory consumption during the design process. A lightweight cipher designer intends to achieve high security and performance algorithms by consuming low memory. We should note that optimizing all these three goals is usually difficult simultaneously and often one of the three designs- security and low cost, security and performance or low costs and performance can be easily optimized [3].

3 The SEPAR Cryptographic Algorithm

Notations used for the description of the SEPAR algorithm listed in Table 1.

Fig. 1 Design strategy of SEPAR

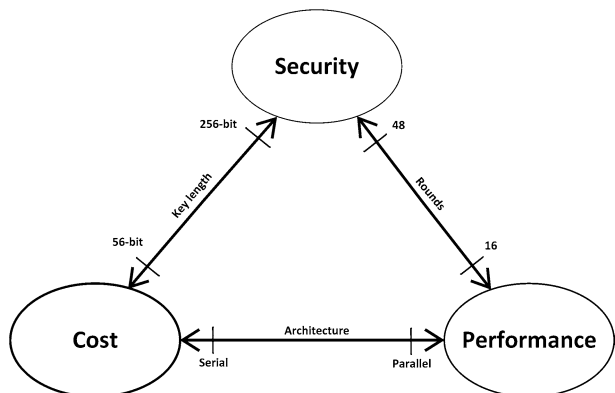


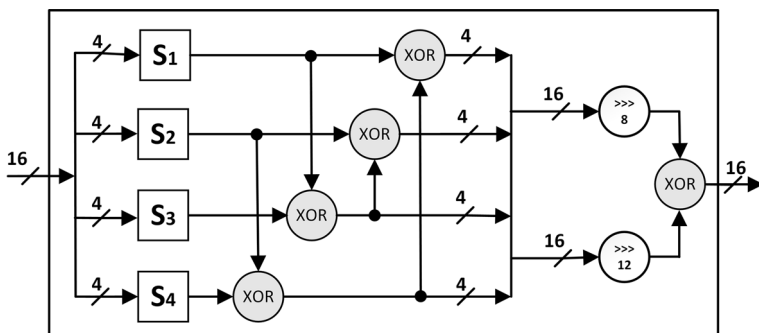
Table 1 Notations used in the description of SEPAR cipher

KEY	The 256-bit master key
K_i	The 32-bit segment key which drives by dividing the master key to 8 equal segments and used in Enc_block_i , $i = 1, 2, \dots, 8$
k_i	THE 16-bit subkey used in each Enc_block , $i = 1, 2, \dots, 6$
State _{i}	The i -th state which i is number, $i = 1, 2, \dots, 9$
PT	The 16-bit plaintext
CT	The 16-bit ciphertext
	Exclusive-or (XOR) operator
	Right circular shift operator, which rotates all bits of its input to the right by n bits
	Left circular shift operator, which rotates all bits of its input to the left by n bits
	Module 2^{16} addition operator
	Module 2^{16} subtraction operator

3.1 16-Bit Block Cipher

SEPAR algorithm employs eight 16-bit block ciphers in its encryption process called Enc_block , in a consecutive manner. In the internal structure of each Enc_block , employed a block encryption function named $b16$. A $b16$ is a typical Substitution-permutation (SP) Network consisting of S-box, XOR, and right circular shift operators. Figure 2 illustrates the structure of the $b16$.

All the employed $b16$ in each Enc_block is composed of lightweight operators such as XOR, circular right rotation, and four meticulously chosen 4×4 S-boxes. The effective integration of these operators leads to a good diffusion and confusion of output property. Each Enc_block consists of four consecutive $b16$ blocks accompanying a layer of 4 S-boxes that functions as a substitution layer. The structure of an Enc_block is shown in Fig. 3.

**Fig. 2** The $b16$ structure

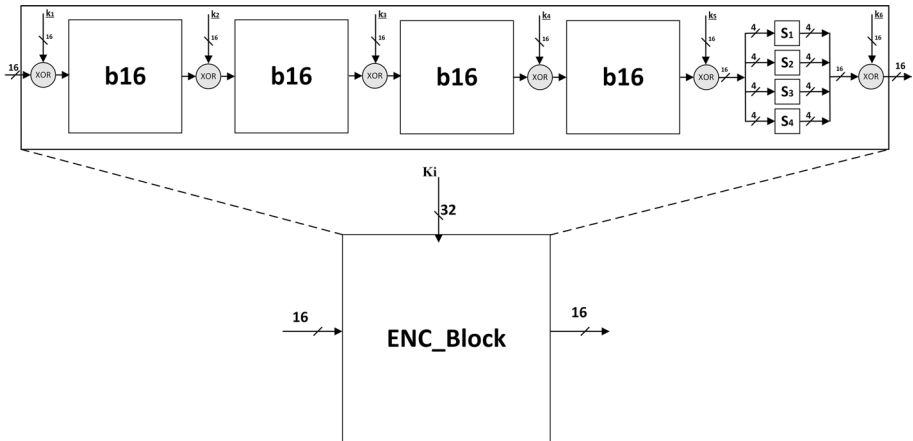


Fig. 3 The Enc_block structure

In an Enc_block, the combination of subkeys and b16s is accomplished at high speed and simply by using XOR operation. The Enc_block operation is described in Algorithm 1.

Algorithm 1: Enc_block operation process

Input: 16-bit input (m) and six 16-bit subkeys (k_i)

Output: A 16-bit data block (m')

```

1: for  $j = 1$  to 4 do
2:    $m \leftarrow m \oplus k_j$ 
3:    $A = m_0 || m_1 || m_2 || m_3$     $B = m_4 || m_5 || m_6 || m_7$ 
    $C = m_8 || m_9 || m_{10} || m_{11}$   $D = m_{12} || m_{13} || m_{14} || m_{15}$ 
4:    $A \leftarrow S(A)$     $B \leftarrow S(B)$     $C \leftarrow S(C)$     $D \leftarrow S(D)$ 
5:    $A \leftarrow A \oplus C$   $B \leftarrow B \oplus D$   $C \leftarrow C \oplus B$   $D \leftarrow D \oplus A$ 
6:    $m \leftarrow A || B || C || D$ 
7:    $m \leftarrow m \oplus (m \lll 8) \oplus (m \lll 12)$ 
8: end for
9:  $m \leftarrow m \oplus k_5$ 
10:  $A = m_0 || m_1 || m_2 || m_3$     $B = m_4 || m_5 || m_6 || m_7$ 
    $C = m_8 || m_9 || m_{10} || m_{11}$   $D = m_{12} || m_{13} || m_{14} || m_{15}$ 
11:  $m \leftarrow S(A) || S(B) || S(C) || S(D)$ 
12:  $m' \leftarrow m \oplus k_6$ 
13: return  $m'$ 

```

3.2 Initialization Process

The overall structure of the SEPAR initialization algorithm is shown in Fig. 4.

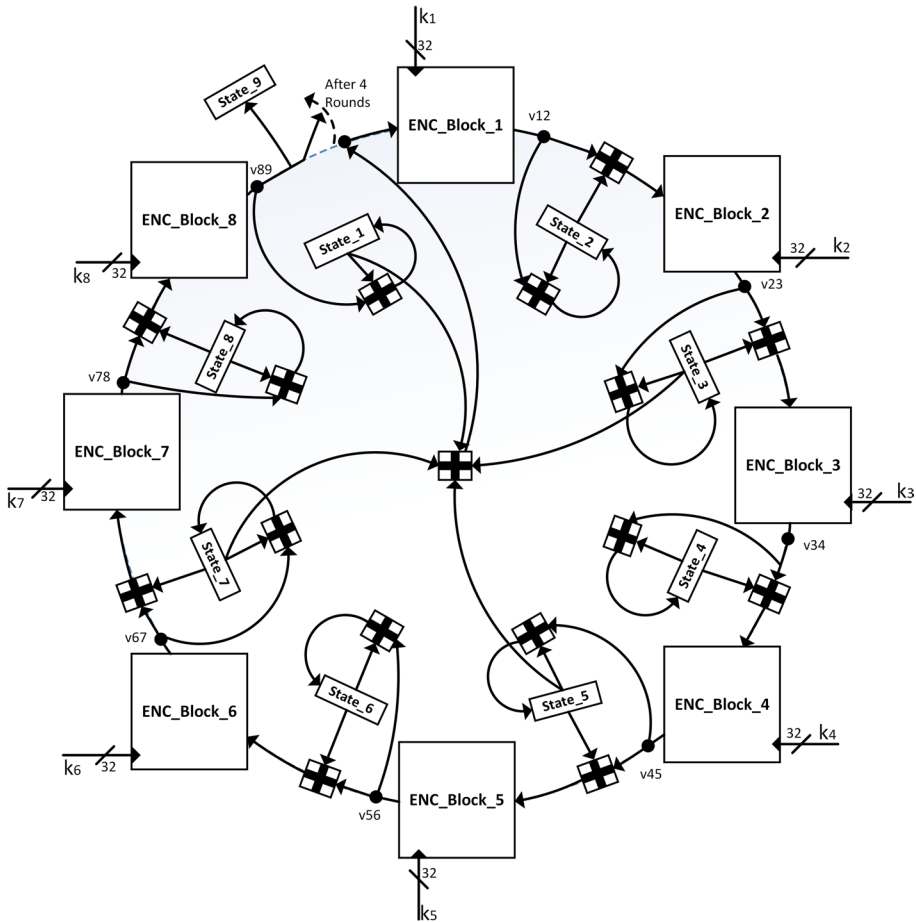


Fig. 4 SEPAR initialization algorithm

Before starting the encryption process, the Initialization algorithm is executed. Eight 16-bit random nonces are chosen for eight internal states $State_i$ ($i = 1, 2, 3, \dots, 8$), respectively, and the Initialization algorithm runs for 4 rounds consecutively. At last, the final output of the fourth round of this execution, stored in $State_9$ which this state used as LFSR in the encryption algorithm. Moreover, the 9th bit of $State_9$ always set to prevent producing an all-zero number. Algorithm 2 shows the initialization process in detail.

Algorithm 2: SEPAR Initialization Algorithm

Input: Eight 16-bit random integers known as a nonce
Output: Initialized eight states State_i ($i=1,2,3,\dots,8$) and LFSR

```

1:  $\text{state}_1 = \text{NONCE}_1$ 
2:  $\text{state}_2 = \text{NONCE}_2$ 
3:  $\text{state}_3 = \text{NONCE}_3$ 
4:  $\text{state}_4 = \text{NONCE}_4$ 
5:  $\text{state}_5 = \text{NONCE}_5$ 
6:  $\text{state}_6 = \text{NONCE}_6$ 
7:  $\text{state}_7 = \text{NONCE}_7$ 
8:  $\text{state}_8 = \text{NONCE}_8$ 
9: for  $t = 0$  to 3 do
10:    $V12_t = \text{Enc\_Block}_{k_1}(((\text{state}_1_t \boxplus \text{state}_3_t) \boxplus \text{state}_5_t) \boxplus \text{state}_7_t)$ 
11:    $V23_t = \text{Enc\_Block}_{k_2}(V12_t \boxplus \text{state}_2_t)$ 
12:    $V34_t = \text{Enc\_Block}_{k_3}(V23_t \boxplus \text{state}_3_t)$ 
13:    $V45_t = \text{Enc\_Block}_{k_4}(V34_t \boxplus \text{state}_4_t)$ 
14:    $V56_t = \text{Enc\_Block}_{k_5}(V45_t \boxplus \text{state}_5_t)$ 
15:    $V67_t = \text{Enc\_Block}_{k_6}(V56_t \boxplus \text{state}_6_t)$ 
16:    $V78_t = \text{Enc\_Block}_{k_7}(V67_t \boxplus \text{state}_7_t)$ 
17:    $\text{Out}_t = \text{Enc\_Block}_{k_8}(V78_t \boxplus \text{state}_8_t)$ 
18:    $\text{state}_{1_{t+1}} = \text{state}_{1_t} \boxplus \text{Out}_t$ 
19:    $\text{state}_{2_{t+1}} = \text{state}_{2_t} \boxplus V12_t$ 
20:    $\text{state}_{3_{t+1}} = \text{state}_{3_t} \boxplus V23_t$ 
21:    $\text{state}_{4_{t+1}} = \text{state}_{4_t} \boxplus V34_t$ 
22:    $\text{state}_{5_{t+1}} = \text{state}_{5_t} \boxplus V45_t$ 
23:    $\text{state}_{6_{t+1}} = \text{state}_{6_t} \boxplus V56_t$ 
24:    $\text{state}_{7_{t+1}} = \text{state}_{7_t} \boxplus V67_t$ 
25:    $\text{state}_{8_{t+1}} = \text{state}_{8_t} \boxplus V78_t$ 
26: end for
27: LFSR =  $\text{Out}_3 \mid 0x100$ 
28: return  $\text{state}_{i_7}$  ( $i = 1 \dots 8$ ) and LFSR

```

3.3 Encryption Process

The overall structure of the SEPAR encryption algorithm is shown in Fig. 5.

After completion of the Initialization process, eight initialized states of the Initialization algorithm are mapped exactly to the encryption algorithm states and State_9 is mapped to LFSR of the encryption algorithm. Encryption can perform now easy by feeding 16-bit data to the encryption algorithm and produce corresponding 16-bit ciphertext after the encryption process. Algorithm 3 describes the encryption process in detail.

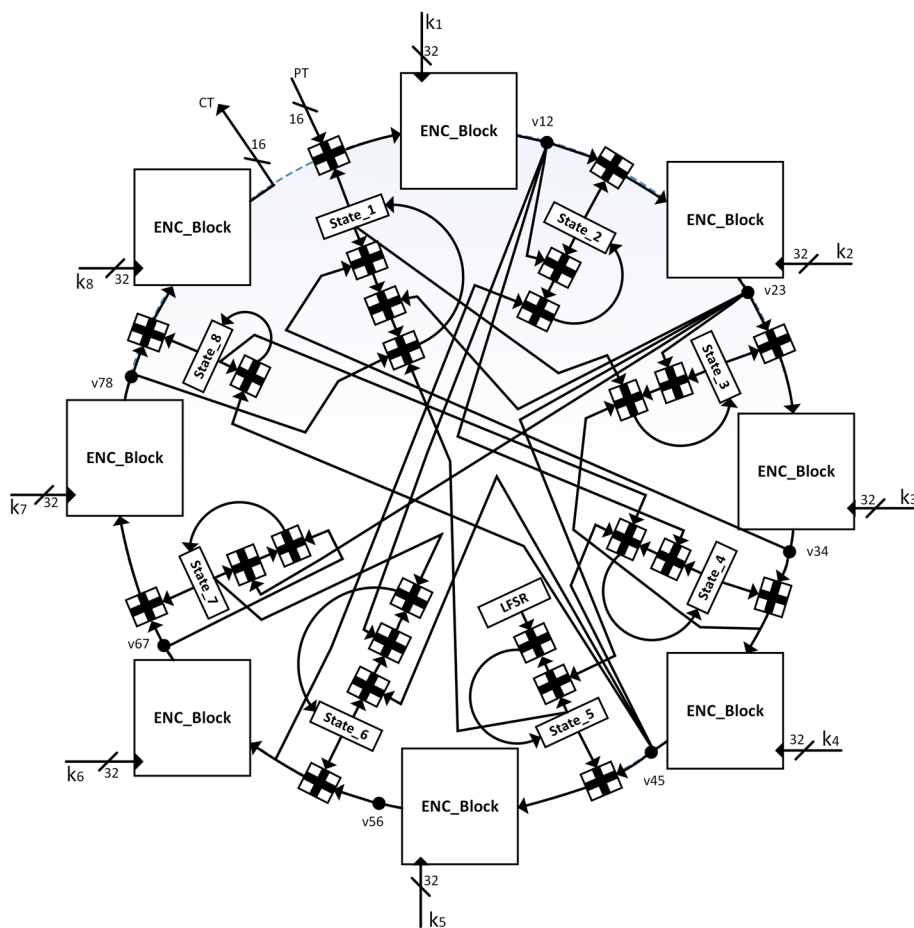


Fig. 5 SEPAR encryption algorithm

Algorithm 3: SEPAR encryption algorithm**Input:** A 16-bit plaintext (PT), eight 16-bit states(State_i) and LFSR**Output:** A 16-bit ciphertext (CT)

```

1:  $V12_t = \text{Enc\_Block}_{k_1}(\text{PT} \boxplus \text{state1}_t)$ 
2:  $V23_t = \text{Enc\_Block}_{k_2}(V12_t \boxplus \text{state2}_t)$ 
3:  $V34_t = \text{Enc\_Block}_{k_3}(V23_t \boxplus \text{state3}_t)$ 
4:  $V45_t = \text{Enc\_Block}_{k_4}(V34_t \boxplus \text{state4}_t)$ 
5:  $V56_t = \text{Enc\_Block}_{k_5}(V45_t \boxplus \text{state5}_t)$ 
6:  $V67_t = \text{Enc\_Block}_{k_6}(V56_t \boxplus \text{state6}_t)$ 
7:  $V78_t = \text{Enc\_Block}_{k_7}(V67_t \boxplus \text{state7}_t)$ 
8:  $\text{CT}_i = \text{Enc\_Block}_{k_8}(V78_t \boxplus \text{state8}_t)$ 
9:  $\text{LFSR}_{t+1} \leftarrow \text{LFSR}_t$ 
10:  $\text{state2}_{t+1} = V12_t \boxplus V56_t \boxplus \text{state6}_t$ 
11:  $\text{state3}_{t+1} = V23_t \boxplus \text{state4}_{t+1} \boxplus \text{state1}_t$ 
12:  $\text{state4}_{t+1} = V12_t \boxplus V45_t \boxplus \text{state8}_t$ 
13:  $\text{state5}_{t+1} = V23_t \boxplus \text{LFSR}_{t+1}$ 
14:  $\text{state6}_{t+1} = V12_t \boxplus V45_t \boxplus \text{state7}_t$ 
15:  $\text{state7}_{t+1} = V23_t \boxplus V67_t$ 
16:  $\text{state8}_{t+1} = V45_t$ 
17:  $\text{state1}_{t+1} = V34_t \boxplus V23_t \boxplus V78_t \boxplus \text{state5}_t$ 
18: return CT

```

3.4 Decryption Process

The overall structure of the SEPAR decryption algorithm is shown in Fig. 6.

In analogous to SEPAR Encryption, the Decryption process is the same as the Encryption process but in the reverse direction. Algorithm 4 describes the encryption process in detail.

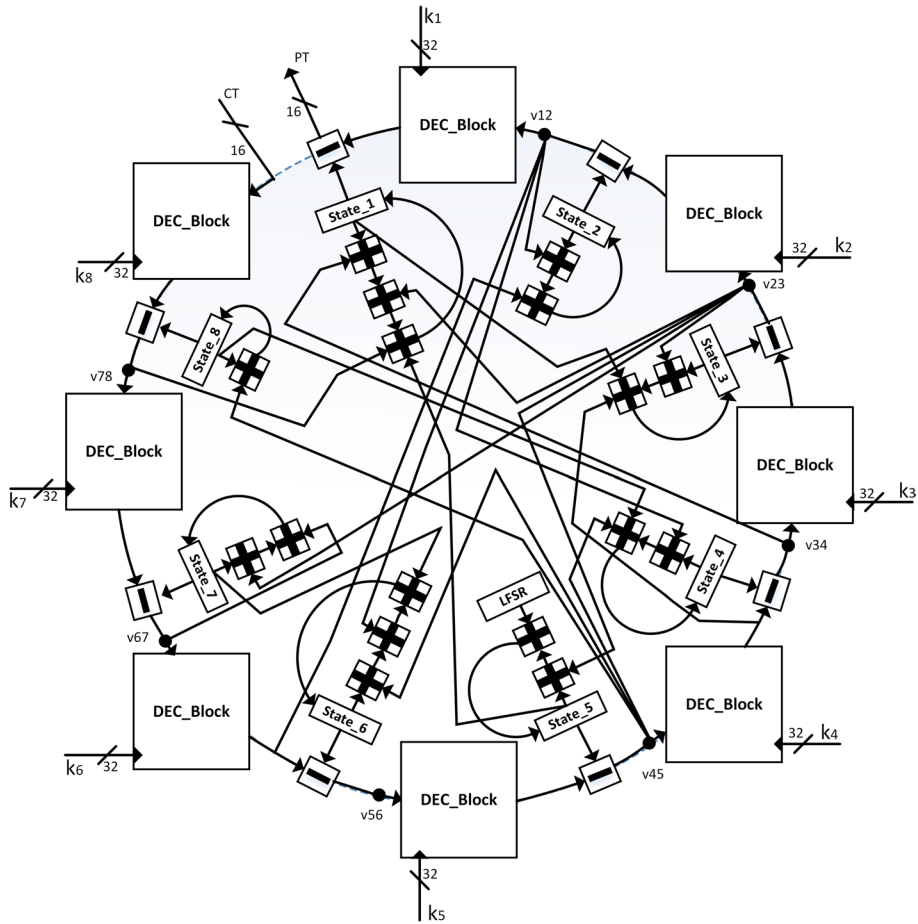


Fig. 6 SEPAR decryption algorithm

Algorithm 4: SEPAR decryption algorithm**Input:** A 16-bit ciphertext(CT) and eight 16-bit states(State_i)**Output:** A 16-bit plaintext(PT)

```

1:  $V78_t = \text{DEC\_Block}_{k_8}(\text{CT}) \boxminus \text{state}_8_t$ 
2:  $V67_t = \text{DEC\_Block}_{k_7}(V78_t) \boxminus \text{state}_7_t$ 
3:  $V56_t = \text{DEC\_Block}_{k_6}(V67_t) \boxminus \text{state}_6_t$ 
4:  $V45_t = \text{DEC\_Block}_{k_5}(V56_t) \boxminus \text{state}_5_t$ 
5:  $V34_t = \text{DEC\_Block}_{k_4}(V45_t) \boxminus \text{state}_4_t$ 
6:  $V23_t = \text{DEC\_Block}_{k_3}(V34_t) \boxminus \text{state}_3_t$ 
7:  $V12_t = \text{DEC\_Block}_{k_2}(V23_t) \boxminus \text{state}_2_t$ 
8:  $\text{PT}_i = \text{DEC\_Block}_{k_1}(V12_t) \boxminus \text{state}_1_t$ 
9:  $\text{LFSR}_{t+1} \leftarrow \text{LFSR}_t$ 
10:  $\text{state}_{2_{t+1}} = V12_t \boxplus V56_t \boxplus \text{state}_{6_t}$ 
11:  $\text{state}_{3_{t+1}} = V23_t \boxplus \text{state}_{4_{t+1}} \boxplus \text{state}_{1_t}$ 
12:  $\text{state}_{4_{t+1}} = V12_t \boxplus V45_t \boxplus \text{state}_{8_t}$ 
13:  $\text{state}_{5_{t+1}} = V23_t \boxplus \text{LFSR}_{t+1}$ 
14:  $\text{state}_{6_{t+1}} = V12_t \boxplus V45_t \boxplus \text{state}_{7_t}$ 
15:  $\text{state}_{7_{t+1}} = V23_t \boxplus V67_t$ 
16:  $\text{state}_{8_{t+1}} = V45_t$ 
17:  $\text{state}_{1_{t+1}} = V34_t \boxplus V23_t \boxplus V78_t \boxplus \text{state}_{5_t}$ 
18: return PT

```

3.5 Key Generator Algorithm

In order to speed up the key generating process and maintain memory performance, subkeys are generated on-the-fly. The SEPAR key Generator design is motivated by Present cipher key-scheduling [4] which has not been reported effective attack so far. Besides, some more complexities such as nonlinearity properties and Enc_block numbers utilized in subkey producing. The key generator algorithm is presented in Algorithm 5.

Algorithm 5: Key Generator Algorithm

Input: A 256-bit master key(KEY) and Number of Enc_block(n)

Output: Six 16-bit Subkeys($\underline{k}_i$)

```

1: KEY = K1 || K2 || K3 || K4 || K5 || K6 || K7 || K8
2: Kj = k1 || k2 . j = 1.2.3.4.5.6.7.8
3:  $\underline{k}_1$  = k1
4:  $\underline{k}_2$  = k2
5:  $\underline{k}_3$  = SBOX((k1 <<< 6)[7.8.9.10]) ⊕ n + 2
6:  $\underline{k}_4$  = SBOX((k2 <<< 10)[7.8.9.10]) ⊕ n + 3
7:  $\underline{k}_5$  = k1 ⊕ k2
8:  $\underline{k}_6$  =  $\underline{k}_3$  ⊕  $\underline{k}_4$ 
9: return  $\underline{k}_i$  . i = 1.2.3.4.5.6

```

3.6 Criteria for Selection of S-Boxes

$S(x) : \mathbb{F}_2^4 \rightarrow \mathbb{F}_2^4$ is 4-bit to 4-bit vectorial Boolean function. S-boxes play an important role in applying nonlinear properties to ciphers, hence the proper choice and design improve their resistance against linear and differential attacks as well as interpolation and cube attacks [11]. SEPAR S-boxes are selected based on investigations proposed in [12, 13]. These four S-boxes are shown in hexadecimal notation in Table 2.

The results of the extensive investigation on SEPAR S-boxes have been summarized in Table 3. These results indicate that important security metrics have been examined to select these four S-boxes.

In order to further investigate selected S-boxes, important and influential security properties for the component functions are also investigated. Suppose $x = (x_3 || x_2 || x_1 || x_0)$ is a 4-bit input of the $S(x)$. An S-box can be defined as:

$$S(x) = S^{(3)}(x) || S^{(2)}(x) || S^{(1)}(x) || S^{(0)}(x) \quad (1)$$

Each $S^{(i)}(x)$ is a component function of $S(x)$. Component functions play an important role in analyzing S-boxes and metrics such as the least algebraic degree and the number of monomial terms is considered important property in cryptanalysis. By regarding the summarized results from Table 4, the algebraic degree of all four component functions in Boolean representation is maximized, the number of monomial terms of each component

Table 2 Selected four S-boxes in hexadecimal notation

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S_1(x)$	1	F	B	2	0	3	5	8	6	9	C	7	D	A	E	4
$S_2(x)$	6	A	F	4	E	D	9	2	1	7	C	B	0	3	5	8
$S_3(x)$	C	2	6	1	0	3	5	8	7	9	B	E	A	D	F	4
$S_4(x)$	D	B	2	7	0	3	5	8	6	C	F	1	A	4	9	E

Table 3 Investigated important properties of selected S-boxes

Property name	$S_1(x)$	$S_2(x)$	$S_3(x)$	$S_4(x)$
Bijective	Yes	Yes	Yes	yes
Balanced	Yes	Yes	Yes	Yes
Involution	No	No	No	No
Linearity weight (Lin(s))	8	8	8	8
Differential order (Diff(s))	4	4	4	4
Nonlinearity indicator	4	4	4	4
Linearity order	4	4	4	4
Linear approximation probability	0.25	0.25	0.25	0.25
Differential approximation probability	0.25	0.25	0.25	0.25
Robustness to differential cryptanalysis	0.750	0.750	0.750	0.750
Algebraic degree(MAX)	3	3	3	3
Algebraic degree(min)	2	2	2	2
Number of fixed points	1	1	2	1
Correlation immunity	0	0	0	0
Algebraic immunity	2	2	2	2
Linear branch number	2	2	2	2
Differential branch number	2	2	2	2

Table 4 Boolean form S-boxes

S-boxes	$S^{(0)}(x)$	$S^{(1)}(x)$	$S^{(2)}(x)$	$S^{(3)}(x)$	Least degree	Total monomial terms
$S_1(x)$	4	5	4	4	3	19
$S_2(x)$	5	6	5	4	3	20
$S_3(x)$	5	4	5	5	3	19
$S_4(x)$	5	5	6	5	3	21

function is small and all component functions have an approximately same number of monomial terms. These properties guarantee the cipher's resistance against attacks that exploit the algebraic properties of S-boxes [14].

By considering Tables 3 and 4 we can conclude that SEPAR S-box selection was done precisely, in order to provide more resistance against cryptographic attacks that exploit S-Box weaknesses.

3.7 Linear Feedback Shift Register(LFSR)

LFSR provides more complexity to the algorithm structure with the simplicity of security analysis meanwhile minimum hardware cost. The polynomial of employed LFSR is defined as followed:

$$f(x) = x^{16} + x^{15} + x^{12} + x^{10} + x^7 + x^3 + 1 \quad (2)$$

This is an irreducible polynomial with a maximum period of $2^{16} - 1$.

4 Security Analysis

In this section, the security analysis of the SEPAR algorithm is presented. The purpose of these analyses is the investigation of cipher resistance against common cryptographic attacks. The combined structure requires us to considering both stream and block cipher attacks.

1. *Birthday attack*: In a birthday attack, the attacker aims to find two groups of identical states, which have initialized with two different IVs and fixed keys. In order to resist this attack, a one-to-one mapping between the initialization algorithm states and the encryption algorithm states should be created [5]. In designing SEPAR cipher, this was considered and a one-by-one mapping is created between the initialization states and the encryption states. Also, the 144-bit length of the internal state of the encryption algorithm prevents exploiting the birthday paradox of finding collision more than 50% [15] due to the complexity of 72-bit testing, hence eight consecutive Enc_block provides enough security.
2. *Differential cryptanalysis*: Enc_block performs encryption operation by a 32-bit segment key. The encryption process is performed in Enc_block with four consecutive b16 with 6 different 16-bit subkeys. For a function $F(x)$ the differential of $F(x)$ and $F(x+a)$ (+ is bitwise addition) is denoted by $D_F(a.b)$ and defined as follows [5, 16]:

$$D_F(a.b) = \left| \{F(x) + F(x+a) = b, x \in \mathbb{F}_2^m\} \right| \quad (3)$$

For examining differential properties of an Enc_block, we compute differentials of Enc_block for a different number of b16. As shown in Table 5, after adding three consecutive b16 to an Enc_block, the differential value remains constant and we get an upper bound of 22. Hence we can conclude that with four consecutive b16 in Enc_block, sufficient security against differential analysis is provided.

For further investigation, the results of an exhaustive search to find differential characteristics with high probability for a different number of b16 are computed and are summarized in Table 6.

As shown in Table 6, seven high-probability differential characteristics are obtained for four consecutive b16. These are listed in Table 7.

The probability value of these 7 differential characteristics is 0.00001907377 which has small significant value in comparison to the ideal probability value of 0.00001525878, hence the attacker cannot exploit these differential characteristics to launch desired attacks. A sample Differential trail with related active S-boxes is illustrated in Fig. 7.

Table 5 Differential property of the 16-bit block cipher

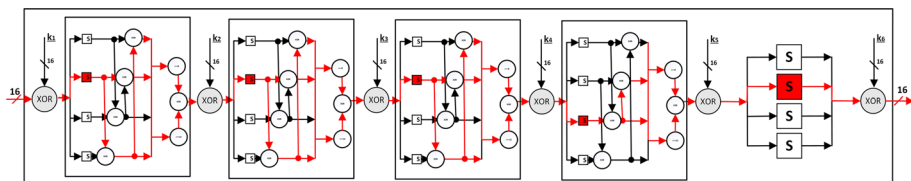
# of b16 in Enc_block	$\max_{a \neq 0, b} D_{b16}(a.b)$
0	16,370
1	1016
2	84
3	22
4	22

Table 6 Higher differential characteristics of Enc_block

# of b16 in Enc_block	# of High probability differential characteristics	Probability value
1	28	$\frac{1}{4}$
2	9	$\frac{1}{64}$
3	7	$\frac{1}{4096}$
4	7	$\frac{1}{52,428}$

Table 7 High-probability differential characteristics for an Enc_block with 4 consecutive b16

#	Input differential characteristic	Output differential characteristic
1	0000, 0100, 0010, 0100	0010, 1010, 0101, 1010
2	0000, 0100, 1001, 0100	0010, 1010, 0101, 1010
3	0000, 0111, 0000, 0100	0101, 1101, 1001, 0011
4	0000, 1011, 0010, 0100	0010, 1010, 0101, 1010
5	0000, 1011, 1001, 0100	0010, 1010, 0101, 1010
6	0000, 1110, 0000, 0100	0101, 1101, 1001, 0011
7	1100, 1100, 1000, 0000	0110, 0001, 1110, 0110

**Fig. 7** A sample differential characteristic trail in an Enc_block

3. *Linear cryptanalysis*: Selected S-boxes used in SEPAR has high nonlinearity properties. In order to investigate the cipher resistance against linear cryptanalysis, the linear property of Enc_block is computed and a distinct bound is defined. If this bound close to a bent function, then we can suggest it is supporting evidence to nonlinearity proof (albeit weak). We note that Boolean bent functions have high nonlinear properties. Bent functions are represented in the form of 2^{2n} . Walsh transform is used to determine the linearity value of a Boolean function. Walsh transform coefficient for bent function is $\pm 2^{\frac{n}{2}-1}$ [5, 17, 18]. The absolute value of the Walsh transform of Enc_block which is denoted by $|Enc_{blockK}(a.b)|$ defined as follows:

$$Enc_{blockK}(a.b) = \sum_{x \in \mathbb{F}_2^{16}} (-1)^{\langle a, ENC_BLOC(x) \rangle + \langle b, x \rangle} \quad a, b \in \mathbb{F}_2^{16}, a \neq b \quad (4)$$

which $\langle \alpha, \beta \rangle$ is the inner product of two binary vectors α and β . Enc_block is a 16-variable Boolean function. The results of Walsh Transform value of Enc_block in a random subset of (a, b, x) with the size of 2^{16} and fixing values of (IV, K) pairs are computed and the results are listed in the Table 8.

According to Table 8, it can be concluded that the Walsh transformation value follows as:

Table 8 Linear properties for a 16-bit Enc_block

$wt(a)$	$wt(b)$	Constant value
1	1	4.890818
1	2	4.502536
1	3	4.474566
2	1	4.790698
2	2	4.336740
2	3	4.790818
3	1	4.939825
3	2	4.655210
3	3	4.335870

$$|Enc_block_K(a.b)| \leq 4.939825 \cdot \left| 2^{\frac{16}{2}-1} \right| \quad (5)$$

The constant 4.939825 is the highest value obtained from Table 8, indicating the bound that an Enc_block can be supposed a bent function. In other words, the computed Walsh transform value could be bounded by $\left| 2^{\frac{16}{2}-1} \right|$ multiplying by a constant. Hence we can conclude that the Enc_block is close to a Bent function and can be resistant to the linear cryptanalysis attack.

4. *Algebraic attack*: The 4-bit to 4-bit S-box can be described with a minimum of 21 equations. In order to completely represent an algorithm algebraically, $x = a \times 21$ quadratic equations in $y = a \times 8$ variables (a represents the number of S-boxes used in the encryption and key scheduling algorithm) are used to examine the cipher [19]. In the SEPAR encryption algorithm, each Enc_block uses 18 S-boxes accompanying 2 more S-boxes for generating third and fourth subkeys. For the whole of 16 Enc_block used in the encrypting process (initialization plus encryption phase), $16 \times 18 = 288$ S-boxes were used and $16 \times 2 = 32$ S-boxes used for the third and fourth subkey generating. The number of quadratic equations is given as:

$$a = (288 + 32) \times 21 = 6720 \quad (6)$$

And the number of variables is given as:

$$b = (288 + 32) \times 8 = 2560 \quad (7)$$

Table 9 summarizes the computed number of the S-boxes, quadratic equations and variables for existing ciphers.

Requiring 6720 quadratic equations and 2560 variables as well as involving internal state transition module 2^{16} operation, it can be assured that applying efficient lineariza-

Table 9 Computed number of S-boxes, quadratic equations and variables

Cipher	# of S-boxes	# of quadratic equations	# of variables
Present-80 [4]	527	11,067	4216
BORON-128 [19]	450	9450	3600
SEPAR	320	6720	2560
MIBS-64 [20]	320	6720	2560
KLEIN-80 [21]	240	5040	1920

tion techniques is hard, hence we can believe cipher shows resistance against algebraic attacks.

5. *Slide and related-key attack*: Exploiting the weakness of the key generator leads to Slide attacks [22, 23] and Related-Key [24, 25] attacks. SEPAR key Generator is motivated by the Present cipher key scheduling design which has not been reported an effective attack so far. Also, some complexities have been added such as applying nonlinearity and affecting by round number to the produced subkeys. Therefore, we can deduce that these added complexities prevent applying these attacks to this SEPAR proposed key generator.
6. *High-order interpolation and differential attack*: These attacks can be applied to block ciphers with a low algebraic degree [26, 27]. As discussed before in the S-box selection subsection, Selected S-boxes have a high algebraic degree. Therefore, these attacks are difficult to apply.
7. *Complementation properties*: If C is the ciphertext of the plaintext P under key K, then $\bar{C}$ is the ciphertext of $\bar{P}$ under key $\bar{K}$ where $\bar{C} \cdot \bar{P}$ and $\bar{K}$ is the bitwise complement of C, P and K [5]. This weakness is not occurring due to the carry propagation between 16-bit states.
8. *Key collision attack*: The majority of symmetric-key ciphers are vulnerable to brute force attacks. Due to the Birthday problem, collision attacks are much faster than brute force attacks. A key collision attack can be applied to any block cipher and depends only on the key length regardless of the key scheduling algorithm [28]. The complexity of creating a message for this attack is $2^{\frac{k}{2}}$ (k is the key length). For SEPAR this complexity of the created message is given as $2^{\frac{256}{2}} = 2^{128}$.
9. *Avalanche effect*: The avalanche effect metric can be used to test the efficiency of the diffusion mechanism. A single bit change in input results in a considerable change in output. The result of evaluating this metric is summarized in Table 10. As it is shown, changes made by one-bit change of plaintext, IV or key bits lead to more than half of bits of ciphertext bits.
10. *Statistical properties analysis*: various statistical tools have been utilized to examine and analyze the statistical properties of SEPAR cipher. NIST Statistical Test Suite is an important testing suite for randomness analysis [29]. NIST considers data to be random if all tests are passed. This suite contains 16 statistical tests for ciphers output randomness testing. Table 11 analysis results of 100 output samples with a length of 10^6 are shown.

A histogram is a chart that shows the frequency distribution of data. In Fig. 8, the histogram of a sample output with the length of 10^6 for two ciphers SEPAR and AES is shown. Analyzing and comparing these two diagrams is a sign of uniform distribution for SEPAR cipher output.

The entropy of a document is an index of its information content and its irregularity. Whatever the entropy of a text rises, we can conclude that it is near to randomness [7]. The entropy calculation can be calculated by:

$$H(x) = - \sum_{i=1}^n p_i \log_2 p_i \quad (8)$$

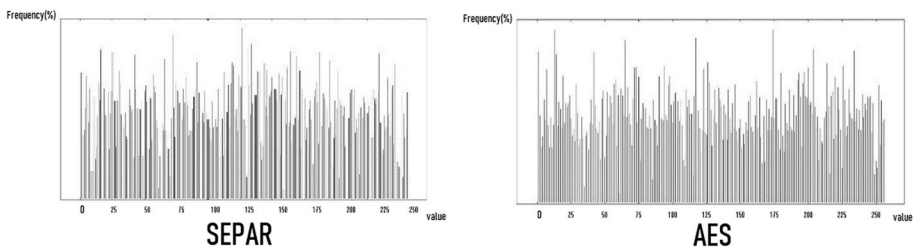
Entropy value for an encrypted custom text with a length of 10^6 and 256 characters is computed and are summarized in Table 12. The values show that SEPAR has the same entropy as well-known existing ciphers.

Table 10 Avalanche effect for SEPAR

	Main test vector	# of bits changed
Plaintext	156F19E18FE6297519A352C45731536A	Base Value
Key	E8B9B733DA5D96D702DD3972E95307FD50C512DBF44A233E8D1E9DF5FC7D6371	Base Value
IV	00000000000000000000000000000000	Base Value
Ciphertext	41E15D769296494746F638CE27FB07E9	Base Value
	Test Vector 1	
Plaintext	056F19E18FE6297519A352C45731536A	One bit Change
Key	E8B9B733DA5D96D702DD3972E95307FD50C512DBF44A233E8D1E9DF5FC7D6371	Fixed
IV	00000000000000000000000000000000	Fixed
Ciphertext	3DA5B84A3909A41592229A0600805A74	# of bits Changed: 70
	Test Vector 2	
Plaintext	156F19E18FE6297519A352C45731536A	One bit Change
Key	E8B9B733DA5D96D702DD3972E95307FD50C512DBF44A233E8D0E9DF5FC7D6371	Fixed
IV	00000000000000000000000000000000	# of bits Changed: 69
Ciphertext	FFC2470821454862CD440E210F2051A7	
	Test Vector 3	
Plaintext	156F19E18FE6297519A352C45731536A	Fixed
Key	E8B9B733DA5D96D702DD3972E95307FD50C512DBF44A233E8D1E9DF5FC7D6371	Fixed
IV	00001000000000000000000000000000	One bit Change
Ciphertext	0B03EF33C4E57C7C6B1E1F31A133DDDD	# of bits Changed: 65

Table 11 Results of NIST statistical analysis on SEPAR

#	Statistical test name	<i>P</i> value	Condition
1	Frequency (Monobit) test	0.487682	PASS
2	Frequency test within a block test	0.20127	PASS
3	Runs test	0.504441	PASS
4	Test the longest run of ones in a block test	0.622270	PASS
5	Binary matrix rank test	0.230424	PASS
6	Spectral DFT test	0.417304	PASS
7	Non-overlapping template matching test	0.513000	PASS
8	Overlapping template matching test	0.588514	PASS
9	Moor's "universal statistical" test	0.249642	PASS
10	Linear complexity test	0.790818	PASS
11	Lempel–Ziv test	0.915995	PASS
12	Serial test	0.703427	PASS
13	Approximation entropy test	0.546145	PASS
14	Cumulative sums (cusum) test	0.697960	PASS
15	Random excursions test	0.506768	PASS
16	Random excursions variant test	0.407024	PASS

**Fig. 8** Histogram of two ciphers SEPAR and AES**Table 12** Computed entropy of SEPAR and existing ciphers on a custom text

Ciphers	Entropy value
SEPAR	7.99
AES	7.99
MARS	7.99
SERPENT	7.99
TWOFISH	7.99
RC6	7.99

5 Efficient Software Implementations

This section presents software implementation results of SEPAR across a wide range of low-power processors used in the IoT environment devices. In our implementation, we used a compact version of the SEPAR cipher wherein, a single S-box is repeated four times instead of using four different S-boxes.

5.1 Efficiency Criteria

As mentioned above, the design process of the SPEAR is to cope with a trade-off between security, cost, and performance. The cost includes evaluating the cipher memory consumption in Flash memory and code size. In order to examine the performance, the throughput criterion is computed. The encryption throughput is the number of encrypted bits at the total encryption time. Throughput defined as follows:

$$\text{Throuput} = \frac{\text{Block Size (bit)}}{\text{Execution Time (ms)}} \quad (9)$$

We should take into account that throughput improvements can be obtained through technics such as parallel processing and pipelining. Throughput improvement leads to power consumption decrement. The security aspect of the ciphers can be computed by the number of key bits.

5.2 Implementation on 8-Bit Atmega128L Microcontroller

In order to implement SEPAR on an 8-bit microcontroller, six subkeys are generated by the key generator on-the-fly which can save the storage requirements. The S-box is also implemented as a 16-bit byte array in which the lower half is used to store the value of S-box and the higher half of a byte is padded with zeroes. Table 13 summarizes the memory consumption and cycle counts for well-known lightweight ciphers as well as SEPAR on the 8-bit microcontroller at 4 MHz.

It is obvious from Table 13 that code size of SEPAR is relatively low than HIGHT cipher which has significantly low cycle counts compared to other ciphers. Cycle counts of SEPAR are approximately two times of the Hummingbird due to employing two more time states in cipher construction. Moreover, there is a significant difference between SEPAR and Present Cycle Counts due to the hybrid structure of SEPAR. In Fig. 9 the overall throughput comparison of existing ciphers on 8-bit microcontroller is shown.

According to the results of Fig. 9, the SEPAR is 3 times faster than the Present, but it is almost 10 times slower than the Hummingbird. The major reason for the speed difference between SEPAR and the Present lies in the combined structure of the cipher. We should take into consideration that SEPAR requires a longer initialization phase compared to the Hummingbird due to utilizing about 2 more internal states. This guarantees the security of SEPAR against cryptography attack which Hummingbird is susceptible [6]. In Tables 14 and 15 execution time of SEPAR in different message lengths at 4 MHz is given.

5.3 Implementation on 16-Bit MSP430 Microcontroller and Development Tools

The 16-bit microcontrollers play an important role in the world of communications and telecommunications due to having appropriate common features of both 8-bit and 32-bit microcontrollers. In Fig. 10, effective parameters of a 16-bit microcontroller that are common between 8-bit and 32-bit microcontrollers are shown.

The 16-bit MSP430 family microcontrollers are widely used in the embedded system design industry due to low power consumption, the flexibility of its peripheral modules and the easy way to use. The MSP430F1611 microcontroller is considered as the

Table 13 Memory consumption and cycle count results on the 8-bit microcontroller

Cipher	Key size [bit]	Block size [bit]	8-bit microcontroller	Code size [Kbytes]	SRAM size [bytes]	int. [cycles]	Enc. [cycles]	Dec. [cycles]
SEPAR	256	16	<i>ATmega128L</i>	3.86	32	39.154	9.761	9.783
Present [4]	80	64	<i>ATmega163</i>	2.39	32		646.166	634.616
DESXL [30]	184	64	N/A	3.19	0	–	8.531	7.961
Salsa20 [30]	128	512	N/A	1.45	280	–	18.400	N/A
HIGHT [30]	128	64	N/A	5.67	0	–	2.964	2.964
Hummingbird [5]	256	16	<i>ATmega128L</i>	3.68	0	14.735	3.664	3.868

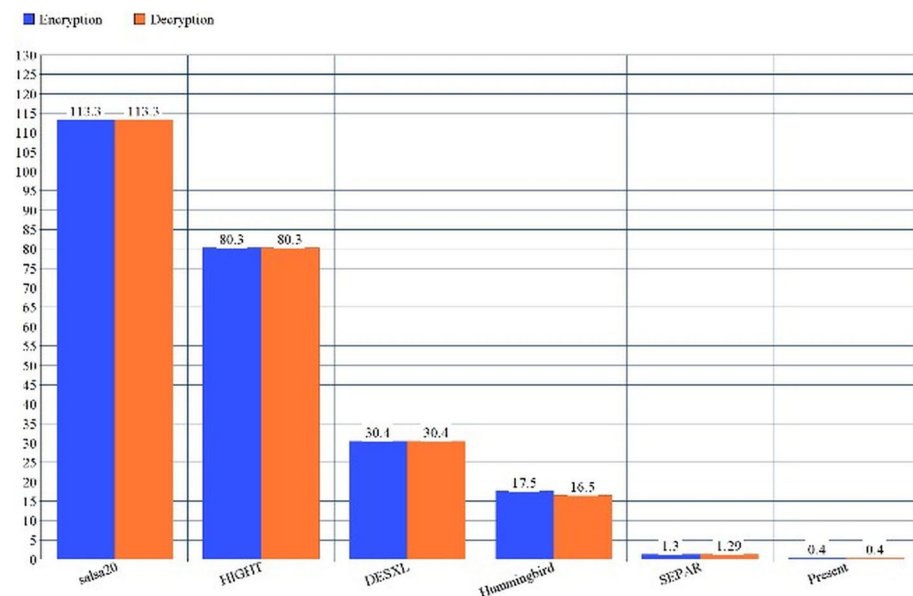


Fig. 9 The overall throughput comparison of existing ciphers on 8-bit microcontroller

Table 14 Encryption execution time of SEPAR in three different data-length at 4 MHz

Message length	Present-enc [4] (ms)	Hummingbird-enc [5] (ms)	SEPAR-enc (ms)	DESXL-enc (ms)	HIGHT-enc (ms)
64-bit	161.54	7.35	19.53	2.10	0.79
128-bit	323.08	11.01	29.32	—	—
192-bit	484.62	14.68	39.15	—	—

Table 15 Decryption execution time of SEPAR in three different data-length at 4 MHz

Message length	Present-dec [4] (ms)	Hummingbird-dec [5] (ms)	SEPAR-dec (ms)	DESXL-dec (ms)	HIGHT-dec (ms)
64-bit	158.65	7.55	19.59	2.10	0.79
128-bit	317.31	11.42	29.53	—	—
192-bit	475.96	15.29	39.29	—	—

implementation and testing purpose of SEPAR cipher. We use CrossWorks for MSP430 version 2 from Rowley Associates to implement and simulate on the target platform. Table 16 summarizes the memory consumption and cycle counts for two well-known lightweight ciphers Hummingbird and Present as well as SEPAR on the 16-bit microcontroller.

We can conclude from Table 16 that Hummingbird has nearly 78% lower memory consumption compared to SEPAR. This difference relies on employed states in cipher structure

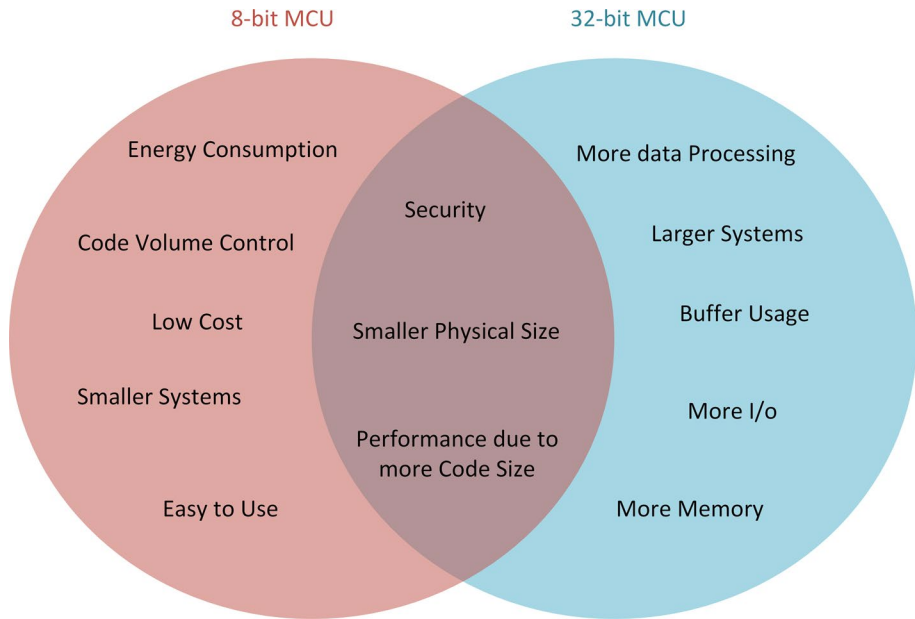


Fig. 10 Featured shared properties of the 16-bit microcontroller

which is nearly two times more than SEPAR, but this improper cost-security trade-off leads to divide and conquer attack [6]. In order to resist this attack, we employed nearly two more states in the SEPAR structure and made a reasonable optimization. Although this trade-off leads to consuming more cycle counts than Hummingbird, it is less than Present. Another reason for this achievement due to the hybrid structure and 16-bit word-internal operation. After the initialization process, SEPAR encryption and decryption algorithms can achieve the throughput of 1.71 Kbps and 1.70 Kbps, respectively. In Fig. 11, the throughput comparison of the SEPAR cipher with lightweight ciphers Hummingbird and Present are shown.

According to the results of Fig. 11, the SEPAR is 9 times faster than the Present, but it is almost 14 times slower than the Hummingbird. The major reason for the speed difference between SEPAR and the Present lies in the combined structure of the cipher. The SEPAR requires a longer initialization phase than the Hummingbird due to using about 2 times more internal states. This guarantees SEPAR security against cryptography attacks which Hummingbird is susceptible [6]. In other words, in hummingbird cipher, the cost-security trade-off did not maintain properly. In Tables 17 and 18 execution time of SEPAR in different message lengths at 4 MHz is given.

5.4 Implementation on 32-Bit ARM LPC2129 Processor and Development Tools

In order to analyze and evaluate the performance of the SEPAR cipher on a 32-bit software platform, LCP 2129 processor is considered. For implementation and test, we use Keil μ Vision that includes an integrated development environment with a standard C language compiler, kernel simulator, and debugging tool. Also in the implementation,

Table 16 Memory consumption and cycle count results at 4 MHz on 16-bit microcontroller

Cipher	Key size [bit]	Block size [bit]	16-bit Microcontroller	Code size [Kbytes]	Int. [cycles]	ENC. [cycles]	DEC. [cycles]
SEPAR	256	16	MSP430F1611	4.67	31,127	7845	7849
present [4]	80	64	C167CR	9.67	–	1,422,565	1,332,062
Hummingbird [5]	256	16	MSP430F1611	2.95	9667	2414	2650

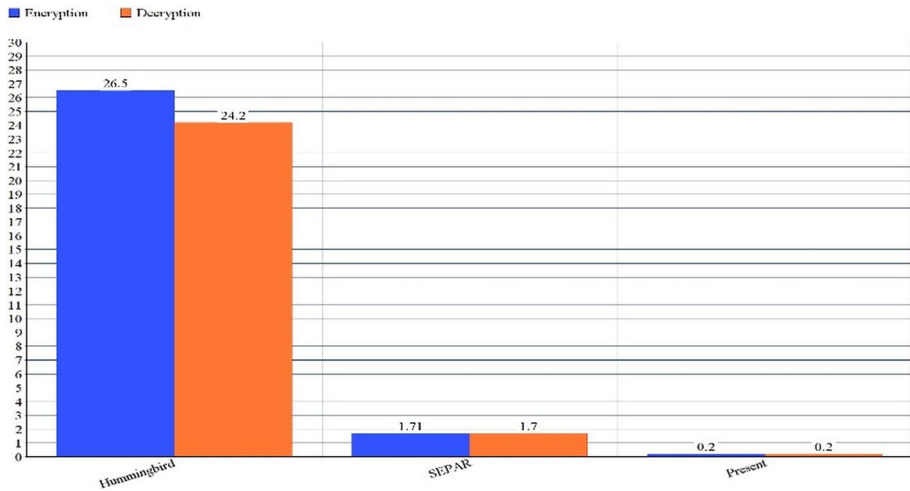


Fig. 11 The Overall throughput comparison at 4 MHz (on 16-bit Microcontroller)

Table 17 Encryption execution time of SEPAR in three different data-length at 4 MHz on 16-bit platform

Message length	Present-enc [4] (ms)	Hummingbird-enc [5] (ms)	SEPAR-enc (ms)
64-bit	360.64	4.83	10.24
128-bit	721.28	7.24	14.78
192-bit	1081.92	9.66	21.53

Table 18 Decryption execution time of SEPAR in three different data-length at 4 MHz on 16-bit platform

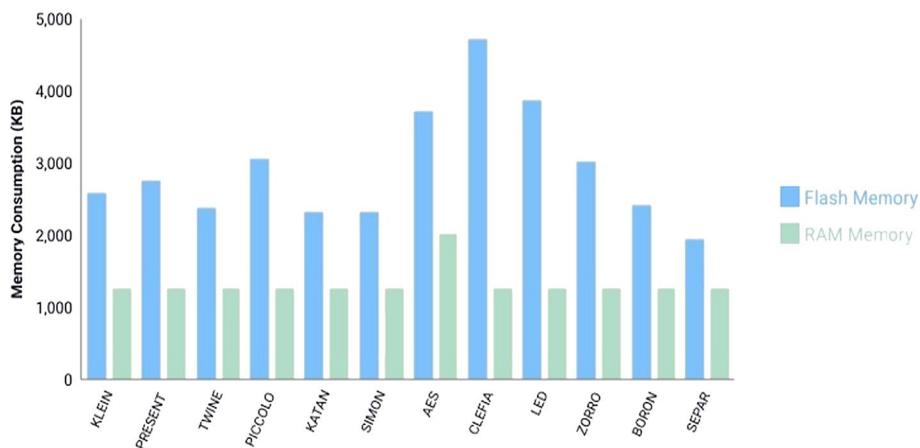
Message length	Present-dec [4] (ms)	Hummingbird-dec [5] (ms)	SEPAR-dec (ms)
64-bit	333.02	5.07	10.91
128-bit	666.03	7.72	15.02
192-bit	1081.92	9.66	21.70

different levels of optimization have been applied. Table 19 summarizes cycle count for existing lightweight ciphers as well as SEPAR on the 32-bit processor [19].

Cycle counts for SEPAR can be seen as a total of the initialization phase and encryption phase. The initialization phase cycle count is 11,256 cycles and the number of cycles required for performing encryption is 2821.21. After the initialization process, SEPAR encryption and decryption algorithms can achieve 2821 cycles, which is less than the majority of existing lightweight ciphers. In the following, a comparison of Flash memory and RAM between standard ciphers and the SEPAR cipher implemented

Table 19 cycle count of well-known ciphers on 32-bit software platform at 12 MHz

Cipher	Structure	Key size [bit]	Block size [bit]	Number of cycles [cycles]
LED	SP-net	128	64	425,572.00
KLEIN	SP-net	96	64	10,650.12
SEPAR	SP-net	256	16	2821.21
BORON	SP-net	128	64	7997.52
Hummingbird2	SP-net	128	16	379,812
Present	SP-net	64	128	3798.12
Speck	Feistel	128	64	588.24
Simon	Feistel	128	64	1268.04
PICCOLO	Feistel	128	64	2732.16
CLEFIA	Feistel	128	128	12,576.12
TWINE	Feistel	128	64	7114.44

**Fig. 12** Flash Memory and RAM comparison in Lightweight Algorithms

on the LPC2129 processor is shown in Fig. 12. All the ciphers as well SEPAR are programmed in embedded C and implemented on a 32-bit processor [19].

Figure 12 shows the required memory size for SEPAR is 2220 as Flash memory and 1256 as RAM. Figure 12 indicates that SEPAR consumes lower memory than other existing ciphers. In order to examine the performance of SEPAR, encryption speed in 64, 128 and 192-bit data lengths are computed and are shown in Table 20.

In Table 21, the results of throughput and execution time of existing lightweight ciphers accompanying SEPAR are presented [19]. Throughput is computed at 12 MHz clock frequency on the software platform.

Table 20 Execution time of SEPAR for three different data-length at 12 MHz

Message length	Execution time (μ s)
64-bit	195.4
128-bit	289.7
192-bit	384.3

Table 21 Results of throughput and execution time at 12 MHz

Cipher	Key size [bit]	Block size [bit]	Execution time [μ s]	Throughput [kb/s]
LED	128	64	7092.86	9.00
KLEIN	96	64	887.51	72.00
BORON	128	64	666.46	96.02
Hummingbird2	128	16	316.51	51.00
SEPAR	256	16	117.308	136.3
Present	64	128	2648.65	24.16
Speck	128	64	49.02	1305.00
Simon	128	64	105.67	605.00
PICCOLO	128	64	227.68	281.00
CLEFIA	128	128	1048.01	122.00
TWINE	128	64	592.87	108.00

Table 22 The overall SEPAR encryption performance improvement at 4 MHz on 8-bit microcontroller

Message length	Performance improvement to present (%)	Performance improvement to hummingbird (%)	Performance improvement to DESXL	Performance improvement to HIGHT
64-bit	87.91	– 165	Less than – 200%	Less than – 200%
128-bit	90.92	– 166	–	–
192-bit	91.92	– 166	–	–

6 Concluding Remarks

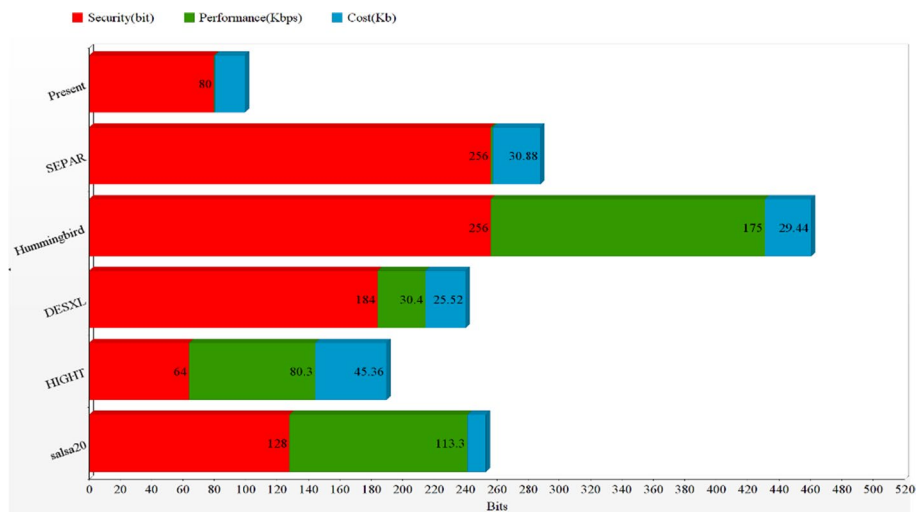
In this paper, a novel lightweight encryption scheme is presented. The design method, which is a smart combination of block and stream ciphers leads to high speed, meanwhile providing cipher robustness and improves cipher resistance against common cryptographic attacks. The overall performance improvement results for some existing ciphers with different message lengths on 8-bit platform are presented in Tables 22 and 23.

Tables 22 and 23 shows that SEPAR achieves around 87.65–91.92% performance improvement when we use SEPAR instead of Present to encrypting and decrypting message blocks with length 64-bit, 128-bit, and 192-bit. Figure 13 compares SEPAR and other existing ciphers. We can conclude that SEPAR coped with the security-performance trade-off on 8-bit microcontroller.

As Fig. 13 indicates, Although SEPAR utilizes a 256-bit key size, it does not provide better performance and also consumes rather more memory than other ciphers. In the

Table 23 The overall SEPAR decryption performance improvement at 4 MHz on 8-bit microcontroller

Message length	Performance improvement to present (%)	Performance improvement to hummingbird (%)	Performance improvement to DESXL	Performance improvement to HIGHT
64-bit	87.65	– 159	Less than – 200%	Less than – 200%
128-bit	90.69	– 159	–	–
192-bit	91.75	– 159	–	–

**Fig. 13** Comparison based on design goals of security, cost, and performance on 8-bit microcontroller**Table 24** The overall SEPAR encryption performance improvement at 4 MHz on 16-bit microcontroller

Message length	Performance improvement to present (%)	Performance improvement to hummingbird (%)
64-bit	97.16	– 112
128-bit	97.95	– 41.43
192-bit	98.01	– 122

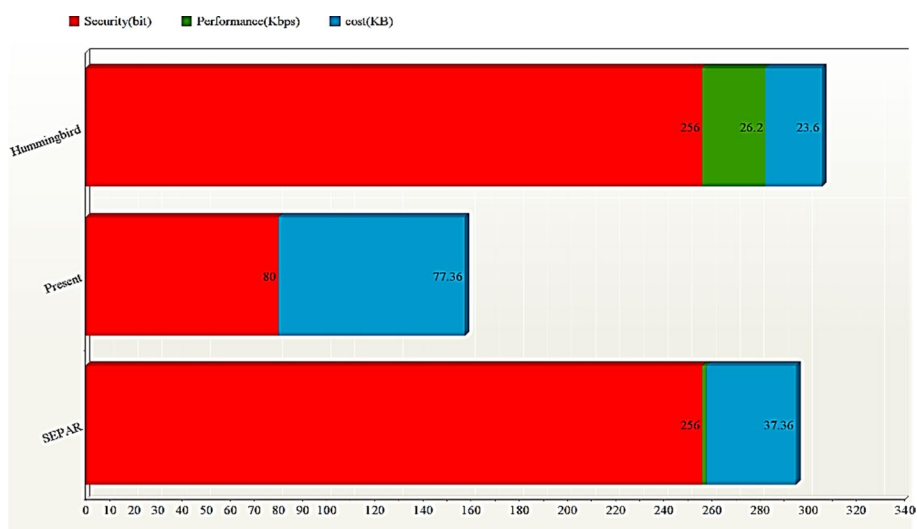
following, the performance improvement results of the SEPAR cipher and the existing ciphers on a 16-bit software platform is shown in Tables 24 and 25.

Tables 24 and 25 shows that SEPAR achieves around 96.72–98.01% performance improvement when we use SEPAR instead of Present to encrypting and decrypting message blocks with length 64-bit, 128-bit, and 192-bit. In Fig. 14 SEPAR and other existing ciphers are compared.

As Fig. 14 indicates, design goals of security, cost, and performance in SEPAR cipher are optimized better than Present because SEPAR utilizes more key-bit size and provides better performance though, consumes less memory. In the other comparison, SEPAR is not as good as Hummingbird. Although Hummingbird utilizes the same key

Table 25 The overall SEPAR decryption performance improvement at 4 MHZ on 16-bit microcontroller

Message length	Performance improvement to present (%)	Performance improvement to hummingbird (%)
64-bit	96.72	– 115
128-bit	97.74	– 94.55
192-bit	97.99	– 124

**Fig. 14** Comparison based on design goals of security, cost, and performance on 16-bit microcontroller**Table 26** The overall SEPAR encryption performance improvement at 12 MHZ on 32-bit ARM processor

SEPAR performance improvement to	Computed performance improvement (%)
Present	464.15
KLEIN	88.93
HUMMINGBIRD2	16.72
BORON	42.22

length size, it provides better performance and consumes less memory than SEPAR. At the end, the performance improvement of the SEPAR and the existing ciphers on 32-bit ARM processor is given in Table 26.

Table 26 shows that SEPAR has remarkably high-performance improvements compared to existing well-known lightweight ciphers such as Present, KLEIN, Hummingbird2, and Boron on a 32-bit software platform. In Fig. 15, SEPAR and other existing ciphers are compared. We can conclude that SEPAR coped with the security, cost, and performance trade-off on a 32-bit microcontroller.

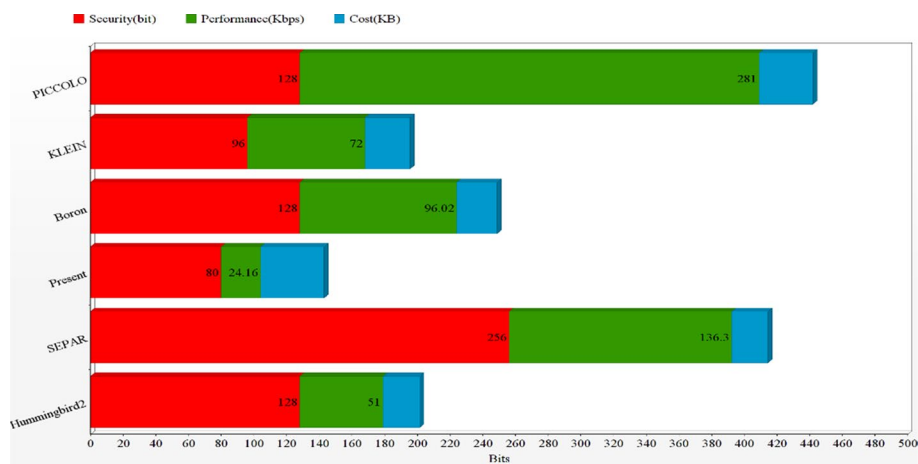


Fig. 15 Comparison of existing ciphers based on design goals of security, cost, and performance

As Fig. 15 indicates, in the SEPAR design all three design goals optimized simultaneously. In other words, by utilizing 256-bit key length and consuming low memory, the better performance provided compared to other ciphers on the 32-bit ARM processor. In overall, the block-stream structure of SEPAR accompanying implementation results on various software platforms gives signs of its well-suitedness for utilizing in the Internet of Things.

References

1. Dabbagh, M., & Rayes, A. (2019). Internet of things security and privacy. *Internet of things from hype to reality* (pp. 211–238). Cham: Springer.
2. Yılmaz, H. E., Sirel, A., & Esen, M. F. (2019). The impact of internet of things self-security on daily business and business continuity. In *Handbook of research on cloud computing and big data applications in IoT* (pp. 481–498). Harrisburg: IGI Global.
3. Poschmann, A. Y. (2009). Lightweight cryptography: Cryptographic engineering for a pervasive world. Ph.D. Thesis, Ruhr-University Bochum, Germany.
4. Bogdanov, A., et al. (2007). PRESENT: An ultra-lightweight block cipher. In P. Paillier & I. Verbauwhede (Eds.), *Cryptographic hardware and embedded systems* (Vol. 4727), CHES 2007. Lecture notes in computer science Berlin: Springer.
5. Engels, D., Fan, X., Gong, G., Hu, H., & Smith, E. M. (2010). Hummingbird: Ultra-lightweight cryptography for resource-constrained devices. In R. Sion, et al. (Eds.), *Financial cryptography and data security* (Vol. 6054), FC 2010. Lecture notes in computer science Berlin: Springer.
6. Saarinen, M. J. O. (2011). Cryptanalysis of hummingbird-1. In *International workshop on fast software encryption*. Berlin: Springer.
7. Boneh, D., & Shoup, V (2015). A graduate course in applied cryptography. *Draft 0.2*.
8. Biryukov, A. (2004). Block ciphers and stream ciphers: The state of the art. *IACR Cryptology ePrint Archive*, 2004, 94.
9. Manifavas, C., et al. (2016). A survey of lightweight stream ciphers for embedded systems. *Security and Communication Networks*, 9(10), 1226–1246.
10. Mohd, B. J., et al. (2018). Hardware design and modeling of lightweight block ciphers for secure communications. *Future Generation Computer Systems*, 83, 510–521.
11. Dinur, I., & Shamir, A. (2009). Cube attacks on tweakable black box polynomials. In *Annual international conference on the theory and applications of cryptographic techniques*. Berlin: Springer.
12. Saarinen, M. J. O. (2011). Cryptographic analysis of all 4×4 -bit s-boxes. In *International workshop on selected areas in cryptography*. Berlin: Springer.

13. Leander, G., & Poschmann, A. (2007). On the classification of 4 bit S-boxes. In C. Carlet & B. Sunar (Eds.), *Arithmetic of finite fields* (Vol. 4547), WAIFI 2007. Lecture notes in computer science Berlin: Springer.
14. Jakobsen, T., & Knudsen, L. R. (2001). Attacks on block ciphers of low algebraic degree. *Journal of Cryptology*, 14(3), 197–210.
15. Kirchner, P. (2011). Improved generalized birthday attack. *IACR Cryptology ePrint Archive*, 2011, 377.
16. Heys, H. M. (2002). A tutorial on linear and differential cryptanalysis. *Cryptologia*, 26(3), 189–221.
17. Tang, C., et al. (2017). Complete characterization of generalized bent and 2 k-bent Boolean functions. *IEEE Transactions on Information Theory*, 63(7), 4668–4674.
18. Carlet, C., et al. (2006). Algebraic immunity for cryptographically significant Boolean functions: Analysis and construction. *IEEE Transactions on Information Theory*, 52(7), 3105–3121.
19. Bansod, G., Pisharoty, N., & Patil, A. (2017). BORON: An ultra-lightweight and low power encryption design for pervasive computing. *Frontiers of Information Technology & Electronic Engineering*, 18(3), 317–331.
20. Izadi, M., Sadeghiyan, B., Sadeghian, S. S., & Khanooki, H. A. (2009). MIBS: A new lightweight block cipher. In J. A. Garay, A. Miyaji, & A. Otsuka (Eds.), *Cryptology and network security* (Vol. 5888), CANS 2009. Lecture notes in computer science Berlin: Springer.
21. Gong, Z., Nikova, S., & Law, Y. W. (2011). KLEIN: A new family of lightweight block ciphers. In *International workshop on radio frequency identification: security and privacy issues*. Berlin: Springer.
22. Biryukov, A., & Wagner, D. (1999). Slide attacks. *International workshop on fast software encryption*. Berlin: Springer.
23. Biryukov, A., & Wagner, D. (2000). Advanced slide attacks. In *International conference on the theory and applications of cryptographic techniques*. Berlin: Springer.
24. Biham, E. (1994). New types of cryptanalytic attacks using related keys. *Journal of Cryptology*, 7(4), 229–246.
25. Ding, L., Jin, C., & Guan, J. (2015). Slide attack on standard stream cipher Enocoro-80 in the related-key chosen IV setting. *Pervasive and Mobile Computing*, 24, 224–230.
26. Jakobsen, T., & Knudsen, L. R. (1997). The interpolation attack on block ciphers. *International workshop on fast software encryption*. Berlin: Springer.
27. Lai, X. (1994). Higher-order derivatives and differential cryptanalysis. *Communications and cryptography* (pp. 227–233). Boston, MA: Springer.
28. Anderson, R., Biham, E., & Knudsen, L. (1998). Serpent: A proposal for the advanced encryption standard. In *First advanced encryption standard (AES) conference*, Ventura, CA.
29. Bassham III, L. E., et al. (2010). Sp 800-22 rev. 1a. A statistical test suite for random and pseudorandom number generators for cryptographic applications.
30. Eisenbarth, T., Kumar, S., Paar, C., Poschmann, A., & Uhsadel, L. (2007). A survey of lightweight-cryptography implementations. *IEEE Design and Test of Computers*, 24(6), 522–533.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Arsalan Vahi was born in 1988, Urmia, Iran. He received his B.S. degree from Payamnoor University in 2017 in software engineering and is currently studying M.Sc. in computer architecture at Islamic Azad University Science and Research Branch. His research interests include Cryptography and Network Security, Wireless Sensor Networks and Internet of things.



Somaye Jafarali Jassbi was born in Tehran, Iran, in 1982. She received the M.Sc. degree in computer architecture engineering in 2007, and the Ph.D. degree in computer architecture engineering in 2010 from the Islamic Azad University Science and Research Branch. In 2010, she joined the Department of computer engineering, Islamic Azad University Science and Research Branch. She became an associate professor in 2011. Her interests is cloud computing, internet of things, wireless sensor network and computer architecture and cryptography. She was head of computer department in 2012. She was also an active member of young researcher club from 2004. She has written, translate and published several professional books and papers in her fields of research.